CAMPUSCLAW · 学期工程

身份认证与知识库问答

结合现有代码的实现方案梳理

本文档把 CampusClaw「第 4 课」交付物中的两条主线拆开讲清楚：**登录态是怎么建立、携带与校验的**，以及**一条提问是怎么被检索证据支撑、再交给生成模型作答的**。所有描述都以仓库中真实存在的代码与规约为依据，并给出可复核的验证结论。

项目	CampusClaw · 班级知识库与教学助手（学期版）
代码基线	`campusclaw-semester`，本地提交 `6d1a699`（分支 `main`）
技术栈	CPython 3.13 · Flask 3.x · SQLite 3（标准库）· bcrypt · Qdrant · 课程网关（OpenAI 兼容）
规约来源	`openspec/changes/add-traceable-vector-retrieval/`（proposal · design · spec · tasks）
文档版本	v1.0
日期	2026-10-13

会话签名令牌 · 班级强隔离 · 三模式检索 · RRF k=60 · 阈值 0.35 · 有依据才生成 · 可溯源引用

目录

1 文档说明

1.1 目的与范围

1.2 结论摘要

1.3 术语约定

2 系统总览

2.1 进程与部署结构

2.2 端到端链路

2.3 模块职责划分

3 身份认证实现方案

3.1 设计约束：为什么是服务端会话

3.2 口令凭据的存储

3.3 会话令牌：签名 Cookie

3.4 登录 / 登出时序

3.5 会话端点契约

3.6 鉴权与授权机制

3.7 班级隔离：唯一授权来源

3.8 会话令牌与 Bearer Token 的取舍

3.9 安全核对清单

4 知识库问答实现方案

4.1 总体原则：先检索，后生成

4.2 资料入库与索引流水线

4.3 切分策略与坐标系统

4.4 索引生命周期与两阶段事务

4.5 三模式检索

4.6 融合公式与阈值

4.7 向量库集成要点

4.8 课程网关适配层

4.9 问答生成：上下文与提示

4.10 无命中纪律

4.11 引用一致性与溯源

4.12 降级与错误码矩阵

5 接口契约总表

6 验证与证据

6.1 测试矩阵

6.2 验证结果

6.3 结论边界

7 附录

7.1 环境变量

7.2 数据表结构

7.3 固化常量

1 文档说明

1.1 目的与范围

本文档回答两个问题：

1. **身份认证是怎么做的**——登录凭证长什么样、存在哪、谁来签发、每一步如何校验，以及「班级」这条安全边界在哪些位置被反复确认。
2. **知识库问答是怎么做的**——一份材料从上传到能被检索经历了什么，一次提问如何被转换成「有证据支撑的回答」，以及在没有证据时系统如何克制自己。

范围限于第 4 课交付物所覆盖的代码，不含前端工程化、流式输出、跨班检索等被规约明确排除在外的能力。

> **阅读须知**
> 文中所有行为描述都以仓库中的代码、规约为唯一依据。代码路径均以 `campusclaw-semester/` 为根；引用规约时给出 `openspec/` 下的文件位置，不引用任何聊天记录。

1.2 结论摘要

签名会话令牌
登录态由 HttpOnly Cookie 承载，不使用 Bearer Token

三道班级闸门
SQL 首谓词 + 向量 pre-filter + 回表复检

0 命中 → 0 次调用
无证据时不触碰生成模型

- **认证**：采用服务端会话，登录态由一枚经服务端密钥签名的会话 Cookie 承载；口令以 bcrypt 单向哈希入库；未登录访问页面跳转 `/login`，访问 API 得到 401 JSON。
- **授权**：班级范围**只**取自服务端会话，请求中任何位置出现的 `class_id` / `role` / `user_id` 一律丢弃。
- **问答**：严格「先检索、后生成」；检索在**本班前 4 条**切片内进行；无命中时接口直接返回固定文案并保持 HTTP 200，且**不产生任何生成模型调用。**
- **可溯源**：回答中的 `[n]` 与 `citations[]` 一一对应，越界引用会被剔除并记入日志。

1.3 术语约定

术语	含义
会话令牌	登录成功后由服务端签发、经服务端密钥签名、随请求自动回传的一串文本。本项目即 `campusclaw_session` Cookie 的值。
班级隔离	任何一次数据读写都必须落在登录者所属班级内；跨班访问对外表现为「查不到」，而非「无权访问」。
切片（chunk）	材料正文按策略切开后的最小检索单元，落库于 `knowledge_chunks`，其 `id` 同时是向量库中的点 id。
证据（hit）	一次检索命中的切片及其得分、名次等元信息。问答模块只依据 hit 作答。

降级	嵌入网关或向量库不可用时，关键字检索仍可用，向量相关模式明确失败（503），而不是返回伪造结果。
RRF	Reciprocal Rank Fusion，按「名次」而非「分数」融合多路检索结果，本项目常数 `k=60`。

2 系统总览

2.1 进程与部署结构

应用由 Flask 应用工厂 `app/__init__.py:create_app()` 组装，注册五个 Blueprint：

Blueprint	模块	职责
`main`	`app/main.py`	站点首页与 `GET /health`（免登录探活）
`auth`	`app/auth.py`	页面会话：表单登录、登出、`login_required` / `teacher_required` 装饰器、口令哈希与校验
`api_auth`	`app/api_auth.py`	会话的三个 JSON 端点：`/api/login`、`/api/logout`、`/api/me`
`materials`	`app/materials.py`	材料上传、列表分页、详情、下载、重建索引、向量对照
`search`	`app/search.py`	检索 `/api/search`、问答 `/api/ask`、来源读取、检索页面

外部依赖只有两个：**课程网关**（OpenAI 兼容的 `/embeddings` 与 `/chat/completions`）与**向量库 Qdrant**。两者都通过标准库 `urllib` 直连，不引入第三方 SDK，且**传输层可注入**，这让离线验收成为可能。

> **关键配置纪律**
>
> `SECRET_KEY`（会话签名密钥）**缺失或短于 16 字符时应用直接启动失败**，禁止任何不安全默认值；而网关与向量库的配置**缺失时不影响启动**——此时 `keyword` 模式照常工作，向量相关模式按降级语义返回 503。

2.2 端到端链路

下图把「登录」与「问答」两条链路放在一起，可以直观看到会话令牌在整条链路中的位置：它是**唯一**携带身份信息的载体，而班级范围在进入检索层之前就已经从会话中取出并固定。

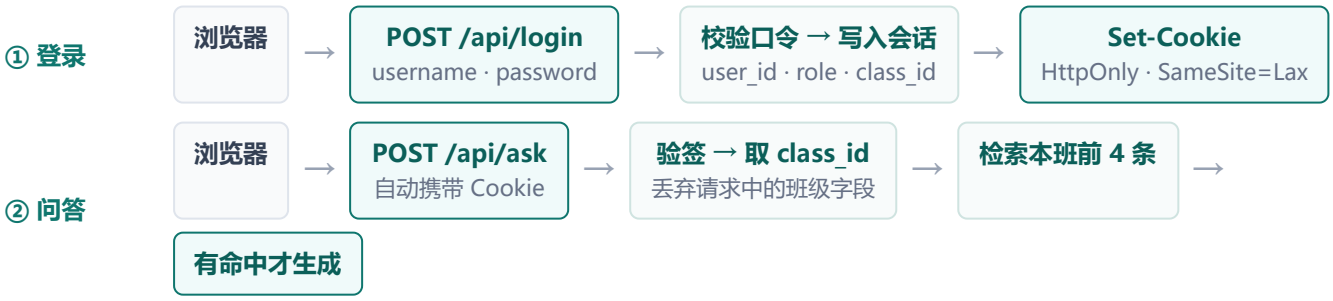


图 2-1　会话令牌贯穿两条链路：签到与校验都用同一枚 Cookie，班级范围只在服务端取一次。

2.3 模块职责划分

认证与问答两条主线的代码落点如下，后续章节按此展开。

文件	职责
`app/auth.py`	口令哈希与校验（bcrypt）、会话写入 `login_session()`、当前用户 `current_user()`、鉴权装饰器、页面登录 / 登出路由

`app/api_auth.py`	会话的 JSON 端点；失败响应同形化，避免账号枚举
`app/db.py`	SQLite 访问层：建表与迁移、关键字检索 SQL、按班级读取的一切查询
`app/chunking.py`	正文预处理（换行归一、空白折叠、链接剥离、NFC）与三种切分策略
`app/indexing.py`	索引编排：切片入库 → 嵌入 → 写向量 → 维护 `index_status`
`app/retrieval.py`	三模式检索、RRF 融合、阈值过滤、输入校验、统一的 hit 数据模型
`app/vector_store.py`	Qdrant 最小子集客户端（集合、写入、删除、带 pre-filter 的检索）
`app/gateway.py`	课程网关适配：嵌入与对话两个接口的请求形状、维度守卫、超时
`app/qa.py`	问答编排：上下文组装、system 提示、引用解析、无命中短路
`app/materials.py`	材料上传流水线（切分 → 落库 → 嵌入）、重建索引、向量对照
`app/search.py`	检索 / 问答 / 来源 / 页面四组 HTTP 端点，负责把领域异常映射为状态码
`scripts/`	建库、建集合、回填索引、规约校验、交付物静态校验

3 身份认证实现方案

本章先说明「为什么是这套方案」，再逐层拆解令牌的形态、签发与校验位置，最后单独讲班级隔离这条安全边界。

3.1 设计约束：为什么是服务端会话

方案不是自由选择的结果，而是规约硬性约束下的唯一解。规约 `openspec/changes/add-traceable-vector-retrieval/specs/traceable-vector-retrieval/spec.md` 中的相关条款原文：

> **Requirement：认证沿用服务端会话**
> 第 4 课 MUST 沿用第 3 课的**服务端会话 Cookie** 认证，MUST NOT 引入客户端持有凭据的登录方案。会话中 MUST 存有该用户的 `class_id`，检索与问答模块 MUST 只从请求上下文读取班级。
>
> **Scenario：会话 Cookie 即登录态** —— WHEN 调用 `POST /api/login` 并成功后查看浏览器存储，THEN 登录态由 `HttpOnly` 会话 Cookie 承载，**AND 响应体与 `localStorage` 中不出现用于后续鉴权的 bearer token。**

这条约束带来三个直接的设计后果：

1. **不使用浏览器端持有凭据的登录方案。**令牌不下发到 JS 可读的位置，因此不存在「前端把令牌存 `localStorage` 再手动塞进 `Authorization` 头」这类写法。
2. **会话必须携带 `class_id`**，且它是后续所有数据访问的授权来源。
3. **检索与问答模块不得自行解析班级**，只能从请求上下文（即会话）读取。

3.2 口令凭据的存储

口令走单向哈希，数据库中没有、也禁止有明文口令列。`app/db.py` 中 `users` 表的定义明确写下了这条约束：

```
CREATE TABLE IF NOT EXISTS users (
  id            INTEGER PRIMARY KEY AUTOINCREMENT,
  username      TEXT    NOT NULL UNIQUE,
  display_name  TEXT    NOT NULL,
  role          TEXT    NOT NULL CHECK (role IN ('teacher','student')),
  class_id      INTEGER NOT NULL REFERENCES classes(id),
  password_hash TEXT    NOT NULL,   -- 仅存单向哈希，禁止明文 password 列（NFR-01）
  created_at    TEXT    NOT NULL
);
```

环节	实现
哈希算法	bcrypt（`app/auth.py` 的 `hash_password()` / `verify_password()`），自带盐与可调代价因子
校验方式	恒定时间比对；不因用户是否存在而改变响应体
角色约束	数据库层 `CHECK (role IN ('teacher','student'))`，非法角色写不进去
批量种子	`app/seed.py` 写入演示账号时同样只存哈希，便于一键复现验收

3.3 会话令牌：签名 Cookie

登录成功后，服务端把身份信息写进 Flask 的 `session` 对象；Flask 会把它序列化、**用 `SECRET_KEY` 签名**，再通过 `Set-Cookie` 交给浏览器。`app/auth.py` 中固化了会话承载的字段集合：

```
SESSION_KEYS = ("user_id", "username", "display_name", "role", "class_id")
```

令牌的形态

令牌（即 Cookie 值）由三段以点号连接而成，这是 `URLSafeTimedSerializer` 的标准输出：



可读性：前两段是 base64 编码，客户端可以解出内容 —— 但改不动第三段。
不可伪造：任何字段改动都会让签名校验失败，服务端随即视为「未登录」。

图 3-1　会话令牌结构：签名保证完整性，但不做加密。

> **这一点必须说清楚**
>
> 会话令牌是**签名**而非**加密**。也就是说，用户能 `base64 -d` 出自己那份 payload，看到 `user_id`、`class_id` 等字段。**这本身不构成漏洞**——因为签名由 `SECRET_KEY` 生成，客户端无法为篡改后的 payload 伪造出合法签名；安全边界建立在「服务端每次都用密钥验签」之上，而不是「用户看不懂内容」之上。
>
> 真正的风险点是 `SECRET_KEY` 泄露：一旦泄露，攻击者可以为任意 `class_id` 签出合法令牌。因此它被强制只能来自环境变量。

Cookie 属性

`app/__init__.py` 的 `create_app()` 中显式设定了四项：

```
SESSION_COOKIE_NAME     = "campusclaw_session",
SESSION_COOKIE_HTTPONLY = True,     # 禁止 JS 读取（document.cookie 取不到）
SESSION_COOKIE_SAMESITE = "Lax",    # 跨站请求不自动携带，缓解 CSRF
SESSION_COOKIE_PATH     = "/",
```

属性	值	作用
`HttpOnly`	True	令牌对 JavaScript 不可见，XSS 无法直接窃取登录态
`SameSite`	Lax	顶层导航之外的跨站请求不携带该 Cookie，降低 CSRF 面
`Path`	/	全站有效
`Secure`	未开启	本地以 HTTP 运行，开发便利优先；**生产部署必须补上**

签名密钥的约束

`_secret_key()` 在应用启动最早期执行校验，不满足条件直接抛 `ConfigError` 终止启动：

```
value = overrides.get("SECRET_KEY") or os.environ.get("SECRET_KEY")
if value is None or not str(value).strip():
    raise ConfigError("缺少环境变量 SECRET_KEY：会话签名密钥仅允许通过服务端环境变量注入…")
if len(value) < MIN_SECRET_KEY_LEN:          # MIN_SECRET_KEY_LEN = 16
    raise ConfigError("SECRET_KEY 长度不足 16 字符，属于不安全默认值…")
```

这样做换来一个明确的失败语义：**配置不当是启动期错误，不会以「能跑但用着不安全」的形态悄悄上线。**

3.4 登录 / 登出时序

#	流向	说明
①	浏览器 → 服务端	提交凭据（JSON 正文或表单，两种方式共用同一套解析）
②	服务端 → SQLite	按 `username` 读取用户行
③	服务端	用 bcrypt 校验口令；**失败时「用户名不存在」与「口令错误」返回同形 401**
④	服务端	`login_session()` 写入会话五个字段
⑤	服务端 → 浏览器	200 + `Set-Cookie`（签名令牌，HttpOnly · SameSite=Lax · Path=/）
⑥	浏览器 → 服务端	后续请求由浏览器**自动**携带 Cookie，前端无需任何代码参与
⑦	服务端	验签 → 读取会话 → 以其中的 `class_id` 作为本次请求的授权范围
⑧	服务端 → 浏览器	登出：`session.clear()` + 令 Cookie 失效；重复调用仍返回 200（幂等）

3.5 会话端点契约

`app/api_auth.py` 提供三个 JSON 端点。它们与页面登录**共用同一套会话写入函数与同一枚 Cookie**，因此页面登录后调 API 同样被认可，反之亦然。

方法	路径	鉴权	行为
`POST`	`/api/login`	免登录	成功 200，返回画像并下发会话 Cookie；失败 **401** 且**不下发**会话
`POST`	`/api/logout`	免登录	清空会话并让 Cookie 失效；**重复调用仍 200**
`GET`	`/api/me`	需登录	返回当前会话画像；未登录 401
`GET` / `POST`	`/login`	免登录	页面登录表单；成功后 302 跳转
`GET` / `POST`	`/logout`	免登录	页面登出，清空会话并重定向

成功响应（`/api/login`、`/api/me`）

```
{
  "user_id": 3, "username": "student01", "display_name": "学生甲",
  "role": "student", "class_id": 1, "class_name": "高一（1）班",
  "ok": true            // /api/me 为 "authenticated": true
}
```

画像中的 `class_id` / `class_name` 来自服务端会话与数据库，**不接受客户端输入覆盖。**

失败响应：同形化

> **反账号枚举**
>
> 「用户名不存在」与「口令错误」返回**完全相同的状态码与响应体**，不泄露账号是否存在，也不回显任何哈希片段：
>
> ```
> HTTP 401
> {"error": "invalid_credentials", "message": "用户名或密码不正确"}
> ```

3.6 鉴权与授权机制

`app/auth.py` 提供两个装饰器，按「谁可以调用」把端点分成两档：

装饰器	语义	典型使用
`login_required`	会话有效即可，教师与学生均可	检索、问答、来源读取、材料列表与下载
`teacher_required`	会话有效且 `role == 'teacher'`	材料上传（表单与 API 两个入口）

两类失败响应：按请求形态分流

`is_api_request()` 判断当前请求是否为接口调用，据此返回不同的失败形态——这直接对应规约 FR-01 S3：

请求类型	状态码	响应
页面请求	`302`	重定向至 `/login`，浏览器自然回到登录页
API 请求	`401`	JSON `{"error": "unauthorized", …}`，前端据此清理并跳登录页
角色不符	`403`	JSON，附 `required_role` 与 `your_role`，便于定位

免登录的例外只有一个：`GET /health`，它既不需要会话也不参与重定向，用于容器探活。

会话指向已删除用户

若验签通过但 `user_id` 在库中已不存在（例如管理员删号），`/api/me` 会**清空会话并按未登录处理**，避免留下悬空登录态。

3.7 班级隔离：唯一授权来源

这是整套方案里最需要被反复确认的一条边界。设计原则一句话：**班级范围只在服务端会话里存在，其他任何地方传来的都不可信。**

第一道：丢弃客户端声明

`app/search.py` 明确列出可能被客户端用来「声明」班级的字段，读到即丢弃并留审计日志：

```
IGNORED_CLIENT_FIELDS = ("class_id", "class", "classId", "role", "user_id", "userId")

def _session_scope() -> tuple[int, str]:
    """唯一授权来源：会话。"""
    return int(session["class_id"]), str(session["role"])
```

查询字符串、JSON 正文、表单中出现的上述字段全部被忽略；响应因此与「不携带这些字段时」**逐字段相同。**

第二道：SQL 首条谓词即班级

关键字检索的 SQL 常量把班级条件放在最前、且是单表谓词，匹配与计数都发生在班级范围之内：

```
SQL_KEYWORD_SEARCH = """
SELECT c.id, c.material_id, c.chunk_index, c.body_text, c.start_offset, c.end_offset,
       c.line_start, c.line_end, c.char_count, c.chunk_sha256, c.source_sha256,
       m.title AS material_title, m.ext AS material_ext
  FROM knowledge_chunks c
  JOIN materials m ON m.id = c.material_id
 WHERE c.class_id = ?                                -- ① 班级边界：最先、单表谓词
   AND instr(lower(c.body_text), lower(?)) > 0       -- ② 匹配：仅在班级范围内
"""
```

按 id 读取切片同样带双条件，使「跨班」与「不存在」在 SQL 层就不可区分：

```
-- SQL_CHUNK_BY_ID
 WHERE c.id = ? AND c.class_id = ?      -- 双条件：跨班与不存在一律查不到
```

审计场景另有一个不带班级过滤的 `get_chunk_any()`，其注释写明**调用方不得把结果直接返回给客户端。**

第三道：向量侧 pre-filter + 回表复检

向量检索同样把班级作为前置过滤条件下发，并在拿到候选后**回关系库再带一次班级**，形成双保险：

```
# app/vector_store.py —— 过滤条件随检索请求一起下发
"filter": {"must": [{"key": "class_id", "match": {"value": int(class_id)}}]}

# app/retrieval.py —— 回表时为每一行再确认一次班级
row = db.get_chunk(int(chunk_id), class_id)   # ② 回表双保险：必须再带 class_id
if row is None:
    continue
```

此外阈值在本地再判一次。也就是说，即便向量库返回了越界或低分候选，也无法穿透到最终结果。

跨班的对外语义

场景	对外响应	理由
检索到他班材料（关键字 / 向量 / 混合）	`200` + 空 `hits`	不得用 403 / 404 暗示「这份资料属于别的班级」
按 id 读取他班切片	`404`（与不存在**同形同体**）	两种情形响应完全一致，不构成存在性泄露
来源已过期	`409` `source_stale`	明确失败，且**不返回正文**
未登录	`401`	鉴权失败先于授权判断

> **为什么不用 403**
> 403 是一个**信息量过大**的响应：它等于告诉调用方「这个资源存在，只是不给你」。在班级隔离场景下，这会变成一个存在性探测接口。统一表现为「查不到」，攻击者无法区分「不存在」与「不属于我」。

3.8 会话令牌与 Bearer Token 的取舍

两种方案都能承载身份，差别主要在凭据的**存放位置**与**校验位置**。本项目按规约选了前者，这里把差异列清楚，便于对照理解。

维度	服务端会话（本项目采用）	Bearer Token
凭据形态	签名令牌，放在 `HttpOnly` Cookie 中	不透明串或自包含令牌，放在请求头
JS 可读性	**不可读**，XSS 无法直接窃取	通常存在 JS 可读的存储中，XSS 可窃取
浏览器自动携带	是，无需前端代码参与	否，需前端显式设置请求头
登出即失效	是（清空会话 + 废弃 Cookie）	需配套吊销机制才成立
CSRF 面	存在，已用 `SameSite=Lax` 缓解	较小（不自动携带）
适用场景	同源 Web 应用	跨端 / 第三方 API 调用

结论：本项目是**同源浏览器应用**，且规约要求凭据不下发到 JS 可读位置。会话 Cookie 在这两个前提下更贴合，也让「登出立即失效」天然成立。需要强调的是——**会话令牌本身就是一种 token**，只是它的存放与传输方式不同；所谓「不使用 Bearer Token」，指的是不使用客户端自行持有并手动附加的头式凭据。

3.9 安全核对清单

#	检查项	状态	说明
1	口令不落明文	已满足	数据库仅存 bcrypt 哈希
2	签名密钥不写死	已满足	仅从环境变量读取，缺失即启动失败

3	密钥长度下限	**已满足**	短于 16 字符拒绝启动
4	令牌 JS 不可读	**已满足**	`HttpOnly`
5	跨站不自动携带	**已满足**	`SameSite=Lax`
6	失败响应同形	**已满足**	避免账号枚举
7	班级不可由客户端声明	**已满足**	唯一授权来源为会话
8	跨班不泄露存在性	**已满足**	200 空结果 / 404 同形
9	网关与向量库密钥不下发	**已满足**	仅在服务端环境变量中；配置视图经 `redacted()` 过滤
10	Cookie 仅在 HTTPS 上发送	**待加固**	未配置 `SESSION_COOKIE_SECURE`；生产环境应置为 True
11	会话有效期策略	**待加固**	未显式配置 `PERMANENT_SESSION_LIFETIME`，如需服务端可控过期须补齐

第 10、11 项是「部署侧加固项」而非实现缺陷：本地与容器内均以 HTTP 运行，开启 `Secure` 会导致 Cookie 无法下发。它们已如实登记，避免被误读为「已经做过」。

4 知识库问答实现方案

问答能力可以理解为一条「入库」和一条「取用」的流水线：入库决定资料以什么粒度被记住，取用决定一次提问能拿到哪些证据、以及拿不到证据时怎么办。

4.1 总体原则：先检索，后生成

> **核心纪律**
>
> 回答必须**由本班资料支撑**。因此流程被固定为「先检索证据 → 再交给生成模型」，并且当检索拿不到任何证据时，**生成模型根本不会被调用**——绝不让模型凭参数记忆「猜」一个答案。

这条纪律决定了两个可见的行为：

- 无命中时接口仍返回 **HTTP 200**，`status` 为 `no_evidence`，`answer` 为固定文案「资料中未找到相关内容」，`citations` 为空数组。
- 无命中**不是** 404 或 403——「没找到证据」与「资源不存在」是两种不同的语义，必须可区分。

4.2 资料入库与索引流水线

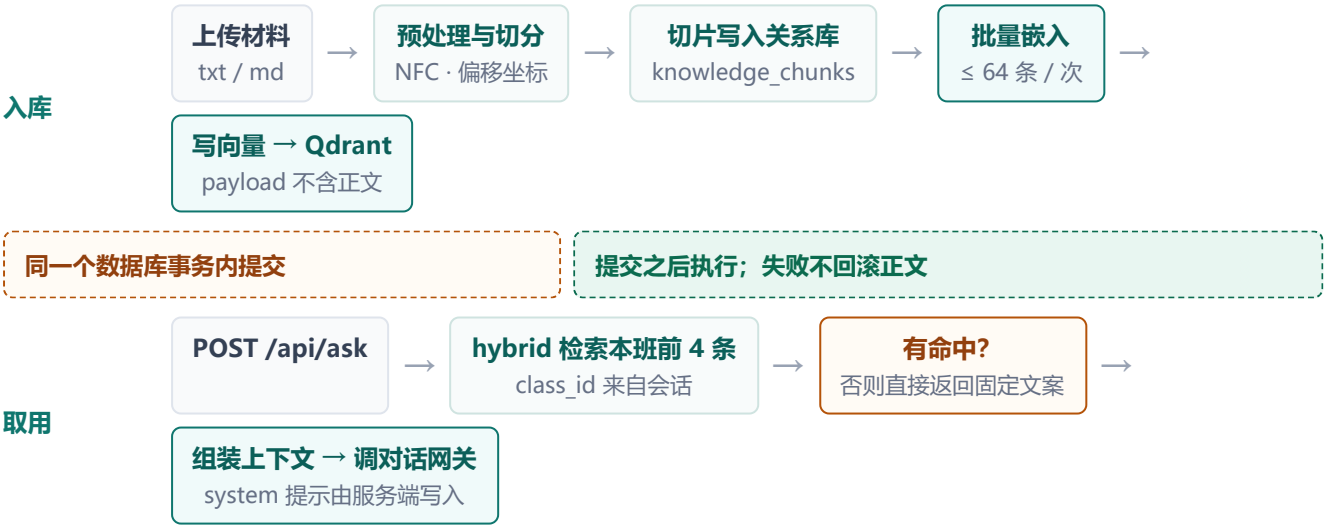


图 4-1　入库流水线（上）与取用流水线（下）。两条色带标出了事务边界的分割点。

4.3 切分策略与坐标系统

切片是检索的最小单元，切得好不好直接决定能不能被命中。策略常量集中在 `app/chunking.py`：

策略	默认参数	行为
`auto`（默认）	上限 800 字，重叠 10%	按「段落 → 行 → 句」的优先级寻找断开点，尽量不切断语义
`custom`	上限 100–2000 字，重叠 0–50%	由调用方指定长度与重叠比例，越界即拒绝
`hierarchy`	继承长度上限	先按标题分章，章内再按长度切分，切片会带上 `section`

常量	值	含义
`DEFAULT_MAX_CHARS`	800	`auto` 的单片长度上限
`DEFAULT_OVERLAP_PERCENT`	10	相邻切片的重叠比例，避免答案正好落在切口上
`MIN_MAX_CHARS` / `MAX_MAX_CHARS`	100 / 2000	`custom` 的长度合法区间
`MIN_OVERLAP_PERCENT` / `MAX_OVERLAP_PERCENT`	0 / 50	重叠比例的合法区间
`MIN_MERGE_CHARS`	32	过短的碎片会被并入相邻切片，避免产生无意义的极短切片
`RANK_PARA` / `RANK_LINE` / `RANK_SENTENCE`	3 / 2 / 1	断开点优先级：段落边界最优，其次行，再次句

预处理：只归一化，不改写

`preprocess()` 依次做四件事，都**不改变文字内容**：换行符归一（CRLF / CR → LF）、连续空白折叠、剥离链接与邮箱、Unicode **NFC** 归一化。NFC 保证「同一个字的不同码点写法」在比较时一致，这直接影响检索命中率。

偏移坐标：一条可复核的约定

每条切片都带 `start_offset` / `end_offset`，用于把命中精确指回原文位置。坐标系统的定义在草案别名端点的响应里显式给出，避免歧义：

```
"coordinate_system": {
  "unit": "unicode_code_point",      // 单位是 Unicode 码点，不是字节
  "normalization": "NFC",            // 已做 NFC 归一化
  "offset_base": "material_full_text",
  "offset_basis": "preprocessed_text",   // 偏移相对「预处理后」的文本
  "snippet_object": "knowledge_chunk.body_text"
}
```

区间约定为**左闭右开**；`line_start` / `line_end` 同样在预处理后的文本上计算，因此 CRLF 与 LF 两种换行下行号一致。

> **由预处理决定的一个后果**
> 偏移是相对**预处理后**文本的，而不是原始上传文件。这是刻意的取舍：它让「偏移 ↔ 切片正文」自洽且可复核，代价是偏移不能直接拿去索引原始文件的第 N 个字节。规约把这条差异如实登记，而不是含糊带过。

4.4 索引生命周期与两阶段事务

索引不是一步完成的，它被拆成两个阶段，中间隔着一个事务提交点。这样拆的理由很实际：**嵌入调用可能失败，但正文不该跟着丢失。**



图 4-2　索引状态机：任何向量侧失败都落到 failed，正文始终保留。

状态	触发时机	可检索性
`pending`	材料已入库但尚未开始索引	取决于切片是否已写入
`indexing`	切片已写入、向量尚未确认	关键字可用
`indexed`	向量写入成功	三种模式全部可用
`failed`	嵌入或写向量失败	**正文与切片保留，关键字仍可命中**；向量侧不可用

两个阶段的职责切分

`app/indexing.py` 把编排拆成三个入口，区别只在「谁掌握事务边界」：

函数	事务边界	使用者
`store_chunks()`	不提交、不回滚，事务属于调用方	上传路径
`index_vectors()`	提交之后执行，各自短事务	上传路径（第二阶段）
`index_material()`	自带完整事务（两阶段串起来）	重建索引、回填脚本

由此得到三条明确的失败语义：

- **切分或写库失败** → 抛异常，整个事务回滚，**不留下任何脏数据**（材料、知识条目、切片同生共死）。
- **嵌入失败** → 材料与切片保留，状态置 `failed`，**不写入不完整的向量**，正文仍可被关键字命中。
- **空正文** → 没有切片可嵌入，直接置 `indexed`，**不调用嵌入网关。**

幂等性如何保证

回填脚本可以反复运行而不产生重复数据，靠的是 `needs_index()` 这个判据：没有切片、或切片记录的 `source_sha256` 与当前正文哈希不一致时才需要索引。

```
def needs_index(chunk_rows, source_sha256: str | None) -> bool:
    rows = list(chunk_rows or [])
    if not rows:
        return True
    return bool(source_sha256) and rows[0]["source_sha256"] != source_sha256
```

重复执行时既不重复切片，也不重复嵌入——这一点在验收中有直接证据（第二次运行 `chunks_written=0、embedded=0、skipped=4`）。

4.5 三模式检索

检索层位于 `app/retrieval.py`，对外暴露三种模式。它们的差别不只是「算法不同」，更在于**是否触碰外部依赖**——这直接决定了降级时的行为。

模式	数据来源	是否调用嵌入网关 / 向量库	降级行为
`keyword`	SQLite 切片表字面匹配	**都不调用**	不受外部依赖影响，始终可用
`vector`	嵌入 + Qdrant 余弦相似度	两者都调用	不可用时返回 503，**不伪造相似度**
`hybrid`	两路融合（RRF）	两者都调用	同上；问答默认使用此模式

关键字路径：为什么用 `instr()` 而不是全文索引

课件给出的参考实现基于 MySQL 的 `FULLTEXT … WITH PARSER ngram`（token 长度 2）。本项目使用 SQLite，没有等价的分词器，因此改用 `instr()` 做字面子串匹配。等价性论证如下：

- 二元 ngram 检索的本质，是判断文本是否包含查询的连续子串；`instr()` 在语义上覆盖了同一个目标，中文连续子串可直接命中，无需分词。
- `lower()` 只对 ASCII 生效，因此「ASCII 大小写不敏感、中文不受影响」正好是想要的语义。
- `instr()` 把 `%`、`_` 当普通字符（不同于 `LIKE`），查询也**不会被当作正则求值**——少了一整类注入与转义问题。

> **如实记录的差异**
> 单字符查询在 ngram 方案下通常因 token 长度下限而不被索引，在 `instr()` 下则可以命中。这是本实现的**能力超出**而非缺失，已在规约中登记，而不是假装两者完全一致。

向量路径：四道关

1. **嵌入**：把查询文本送入嵌入网关，得到查询向量。
2. **带班级的向量检索**：下发 `class_id` pre-filter，同时带 `score_threshold=0.35`。
3. **回表复检**：用切片 id 回关系库取正文，**再带一次** `class_id`，取不到就丢弃。
4. **本地阈值再判**：低于 0.35 的候选在本地再过滤一次。

输入校验

检索参数由 `validate_query()` 统一校验，原则是**不静默改写、不静默截断**：

参数	约束	越界行为
`q`	非空、≤ 300 码点、不含控制字符	400 `invalid_query`
`limit`	1 – 20（默认 10）	400

`offset`	0 – 1000（默认 0）	400
`mode`	`keyword` / `vector` / `hybrid`	400

注意布尔值会被显式拒绝（`limit=true` 不因 Python 里 `True == 1` 而被接受），这是「不静默」原则的一个细节。

4.6 融合公式与阈值

RRF 融合（k = 60）

混合模式不用「分数相加」，而是按名次融合。原因是关键字命中次数与余弦相似度**量纲不同**，直接相加会让某一路主导结果。RRF 只看名次：

```
score_final = 1 / (60 + rank_keyword) + 1 / (60 + rank_vector)

# 缺席的一路不贡献任何分数（不是按 0 分参与相加）
```

举一个具体例子，三个候选切片的融合结果：

切片	关键字名次	向量名次	融合分计算	结果
A	1	3	1/61 + 1/63 = 0.016393 + 0.015873	**0.032266**（最高）
B	2	—	1/62	0.016129
C	—	1	1/61	0.016393

可见**两路都能命中的切片明显占优**，而单路命中的两个候选仍能进入结果——这正是混合检索想要的：既奖励「两条证据线都指向它」，又不丢掉只被一路发现的内容。

阈值：0.35

向量相似度低于 `0.35` 的候选被丢弃，界值本身**保留**（即 `score < 0.35` 才丢）。规则明确到可以写出边界用例：

余弦相似度	是否保留	说明
0.3499	丢弃	严格低于阈值
0.35	**保留**	界值包含在内
0.9	保留	——

排序稳定性

排序键为 `(-score_final, material_id, chunk_index)`。当融合分相同时，用材料 id 与切片序号兜底，保证**同一查询反复执行得到完全相同的顺序**——这对可复核性与分页一致性都是必要的。

4.7 向量库集成要点

`app/vector_store.py` 封装了 Qdrant 的一个最小子集。有三处刻意的设计：

要点	实现
点 id 与业务 id 对齐	Qdrant 点 id 直接使用 `knowledge_chunks.id`，不需要额外的映射表
payload **只含标识**	仅 `chunk_id` / `material_id` / `class_id` / `chunk_index` / `chunk_sha256`，**不含正文**
写入前置断言	`upsert()` 检测到 payload 含 `body_text` 或缺 `class_id` 时直接抛错
检索必带 pre-filter	`search()` 强制构造 `class_id` 过滤条件，调用方无法绕过

> **为什么不把正文存进向量库**
> 两个理由。其一，**单一数据源**：正文只在关系库里存在一份，避免两处不一致；向量库只负责「谁的向量最接近」。其二，**权限边界更简单**：即使向量库被绕过，能拿到的也只是几个整数标识，拿不到任何材料内容。

集合约定为 `campusclaw_chunks`，距离度量为 **Cosine**，`size` 等于 `EMBEDDING_DIM`。集合创建是幂等的，并且在删除旧向量**之前**会先确保集合存在，使「全新部署未手工初始化」的场景也能自愈。

4.8 课程网关适配层

`app/gateway.py` 只做一件事：把领域调用翻译成 HTTP 请求，并把失败翻译回领域异常。

能力	请求形状	约束
嵌入	`POST {base}/embeddings`，正文**只含** `model` 与 `input`	按 ≤ 64 条分批；返回条数必须与输入一致
对话	`POST {base}/chat/completions`，正文含 `model` / `messages` / `temperature`	只取 `choices[0].message.content`；非文本即判失败

维度守卫

嵌入返回的向量长度必须等于 `EMBEDDING_DIM`，否则抛 `EmbeddingDimMismatch`，调用方据此把切片置 `failed` 且**不写入向量**。这条检查挡掉的是最隐蔽的一类故障：维度不匹配的向量能被存进去，但检索结果的相似度会完全失去意义。

密钥边界

密钥只从环境变量读取，通过请求头的 `Authorization: Bearer <key>` 发给网关——注意这是**服务端发往网关**的头，与浏览器的登录方式是两件事。对外暴露的配置视图统一经过 `redacted()` 过滤：

```
def redacted(self) -> dict:
    """可安全输出的配置视图（**不含密钥**）。"""
    return {"embedding_base_url": …, "embedding_model": …, "embedding_dim": …,
            "embedding_ready": …, "chat_model": …, "chat_ready": …}
```

据此可以对外报告「嵌入 / 对话是否就绪」，而不泄露任何密钥。

4.9 问答生成：上下文与提示

问答编排在 `app/qa.py`。它负责把「检索结果」变成「一次模型调用」，并且在这之间不做任何自由发挥。

消息序列

发给对话网关的消息序列**完全由服务端构造**，顺序固定：

序号	角色	内容
1	`system`	服务端写死的答题纪律提示
2	`system`	证据块：本班命中的切片，带编号
3…n	`user` / `assistant`	客户端提供的多轮历史（已清洗）
末	`user`	用户最新一句提问

第 1 条提示的原文如下，它对模型的约束是「只许依据编号资料作答」：

```
你是班级知识库助手。**只允许依据下面带编号的资料作答**，
不得依据你自己的记忆、常识或猜测补充任何事实。
若资料不足以回答，直接回答「资料中未找到相关内容」。
回答必须简短，并在引用处标注编号，格式为 [1]、[2]，编号与资料序号一一对应。
```

第 2 条证据块的格式由 `_context_block()` 生成，**只含材料标题、切片序号与切片正文**：

```
以下是本班知识库中检索到的资料，编号即引用编号：

[1] 材料《记叙文阅读方法》第 3 段：
…（切片正文）…

[2] 材料《记叙文阅读方法》第 4 段：
…（切片正文）…
```

> **上下文里刻意不放的东西**
>
> 不含向量分量、不含 Qdrant 原始点数据、不含其他班级的切片。前两者的排除让「密钥与相似度不进入会话内容」，后者是班级隔离在问答链路上的延伸。

历史消息的处理

规则	实现
丢弃客户端 `system` 消息	只保留 `user` / `assistant`；丢弃时记一条 warning。**system 提示无法被客户端替换或追加**
长度上限	只保留最近 20 轮，避免上下文无限膨胀
检索只用最新一句	检索仅以 `q` 发起；历史只附在对话上下文之后，**不参与检索**
证据条数	最多 4 条（`TOP_K_CONTEXT = 4`）

4.10 无命中纪律

> **本变更最核心的一条**
>
> 当检索结果为空时，`answer()` **直接返回**，不做任何模型调用。代码里的意图注释写得很直接：
>
> ```
> if not hits:
>     # 刻意不调用对话网关：让模型凭参数记忆勉强作答正是课件否决的做法
>     log.info("问答无命中，未调用对话网关（class_id=%s）", class_id)
>     return payload
> ```

对应的响应如下，注意**状态与文案同时给出**，且 HTTP 状态是 200：

```
HTTP 200
{
  "answer": "资料中未找到相关内容",
  "status": "no_evidence",
  "message": "资料中未找到相关内容",
  "citations": [],
  "question": "…",
  "degraded": false,
  "index_warnings": []
}
```

验收对此的判据不是「页面上出现了这句话」——因为页面出现文字并不等于模型被调用过——而是**直接断言对话网关的调用次数为 0**。

4.11 引用一致性与溯源

回答里的引用编号与 citation 列表必须严格对应，否则「可溯源」就是一句空话。

环节	规则
编号来源	组装证据块时按顺序编号 `[1]`、`[2]`…，编号即切片在上下文中的位置
列表顺序	`citations[i]` 对应 `[i+1]`，与实际传给模型的切片顺序一致

字段完整性	每条 citation 含 `chunk_id` / `material_title` / `chunk_index` / `start_offset` / `end_offset` / `snippet`
越界引用	超出实际切片数的编号**不进 citations**，服务端记 warning，也不返回他班或不存在的引用
回溯验证	用 `chunk_id` 访问来源端点，可拿回同一条切片正文

引用解析由一条正则完成，返回「合法编号」与「越界编号」两组，都去重并升序：

```
_CITATION_RE = re.compile(r"\[(\d{1,3})\]")
```

来源读取：重新鉴权 + 版本核对

从引用跳回去读原文时，端点是**独立重新鉴权**的，不依赖前端此前拿到的任何结果：

- 按 `(chunk_id, class_id)` 双条件查询；跨班与不存在返回**同形同体**的 404。
- 比对切片记录的 `source_sha256` 与所属材料当前的 `source_sha256`；不一致说明材料已被重建索引，返回 **409** `source_stale` 且**不返回正文**，提示「请重新检索」。
- 会话失效则 401。

4.12 降级与错误码矩阵

这套错误码的设计目标是：**每一种失败都明确、可区分，且绝不退化成伪造结果。**

状态码	错误码	触发条件	系统行为
400	`invalid_query`	查询为空 / 超长 / 含控制字符 / 参数越界	明确拒绝，不静默截断
401	`unauthorized`	未登录或会话失效	前端清理状态并回到登录页
403	角色不符	非教师调用上传 / 重建索引	附 `required_role` 与 `your_role`
404	`not_found`	来源不存在，或**属于其他班级**	两种情形同形同体
409	`source_stale`	引用来源已被重建索引	不返回正文，提示重新检索
502	`upstream_error`	对话网关超时或报错	失败外抛，**不返回本地拼凑的答案**
503	`embedding_unavailable`	嵌入网关未配置或不可用	向量相关模式失败；`keyword` 不受影响
503	`vector_store_unavailable`	向量库未配置或不可达	同上，**不伪造相似度分数**

> **降级不等于糊弄**
> 降级的正确形态是「**清晰地少做一件事**」，而不是「用本地规则假装做到了」。嵌入不可用时，系统宁可返回 503 说「这条路走不通」，也不会用随机数或字面重合度冒充相似度——因为后者会产出一个看起来正常、实际上不可信的结果。

5 接口契约总表

下表汇总全部端点。除标注「免登录」者外，其余均需有效会话。

方法	路径	鉴权	说明
`GET`	`/health`	免登录	探活；可选检测 SQLite 可读性，失败返回 503
`GET`	`/`	免登录	首页；已登录与未登录展示不同内容
`GET`	`/login`	免登录	登录页
`POST`	`/login`	免登录	表单登录；成功 302
`GET` / `POST`	`/logout`	免登录	页面登出
`POST`	`/api/login`	免登录	JSON 登录；成功下发会话 Cookie，失败同形 401
`POST`	`/api/logout`	免登录	幂等登出
`GET`	`/api/me`	需登录	会话画像
`GET`	`/api/materials`	需登录	材料列表；支持 `page` / `page_size` / `title` 过滤
`GET`	`/api/materials/<id>`	需登录	材料详情；含索引状态、切片数、策略等
`GET`	`/api/materials/<id>/download`	需登录	下载原始文件
`POST`	`/api/materials/upload`	教师	上传并即时索引；返回切片摘要与索引状态
`POST`	`/api/materials/<id>/reindex`	教师	重建索引：先删旧向量再写入
`GET`	`/api/materials/<id>/vectors`	需登录	关系库切片 ↔ 向量库记录对照；**不回传向量分量**
`POST`	`/api/search`	需登录	三模式检索；`mode` 默认 `hybrid`
`POST`	`/api/ask`	需登录	知识库问答；先检索后生成
`GET`	`/api/knowledge/sources/<chunk_id>`	需登录	来源读取；独立重新鉴权 + 版本核对
`GET`	`/api/knowledge/search`	需登录	草案期别名端点，保留旧字段名
`GET`	`/search`	需登录	检索与问答页面

`POST /api/search` 成功响应要点

```
{
  "query": …, "mode": "hybrid",
  "hits": [ { "chunk_id": …, "material_id": …, "material_title": …,
              "chunk_index": …, "start_offset": …, "end_offset": …,
              "snippet": …, "score_keyword": …, "score_vector": …,
              "rank_keyword": …, "rank_vector": …, "score_final": … } ],
  "total_matched": …, "returned": …, "status": "ok",
  "degraded": …, "index_warnings": [ … ], "class_id": …, "role": …
}
```

`query_vector` 字段在服务端结构里存在，但**不进入页面 DOM**；页面只渲染文本类字段。

`POST /api/ask` 成功响应要点

```
{
  "answer": "…带有 [1] [2] 标注的简短回答…",
  "status": "ok",
  "citations": [ { "chunk_id": …, "material_title": …, "chunk_index": …,
                   "start_offset": …, "end_offset": …, "snippet": … } ],
  "referenced": [1, 2], "out_of_range_refs": [],
  "question": …, "degraded": …, "index_warnings": [ … ],
  "class_id": …, "role": …, "top_k": 4
}
```

6 验证与证据

本章给出复核方式与结论，并明确边界——哪些结论是真实环境验证的，哪些是等价替身验证的。

6.1 测试矩阵

测试文件	层次	覆盖内容
`tests/test_chunking.py`	单元	预处理、三种切分策略、偏移与行号、边界、确定性
`tests/test_retrieval.py`	单元	网关配置与请求形状、维度守卫、向量 payload、pre-filter、阈值、RRF 公式、输入校验、排序稳定、坐标系统
`tests/test_qa.py`	单元	上下文组装、system 提示不可替换、只取最新一句、引用顺序、网关失败不外抛伪造答案
`tests/test_search_page.py`	单元	页面转义、会话失效清理、三态文案、过期提示、向量不入页面、禁用凭据存储
`tests/http_retrieval_acceptance.py`	真实 HTTP	本变更全部 Scenario 的正向与反向判据
`tests/test_acceptance.py`	场景	既有能力的回归
`tests/http_acceptance.py`	真实 HTTP	既有能力的真实 HTTP 回归
`scripts/validate_spec.py`	规约	规约结构校验与覆盖检查
`scripts/check_compose.py`	交付物	Docker 交付物静态校验

6.2 验证结果

本次梳理对全部套件重新执行了一遍，结果如下：

套件	结果	说明
`tests/test_chunking.py`	**35 / 35 OK**	预处理、三种切分策略、偏移与行号、边界、确定性
`tests/test_retrieval.py`	**68 / 68 OK**	三模式、RRF、阈值、pre-filter、维度守卫、排序稳定
`tests/test_qa.py`	**28 / 28 OK**	上下文组装、system 丢弃、无命中零调用、引用顺序
`tests/test_search_page.py`	**30 / 30 OK**	转义、三态文案、会话清理、向量不入页面
单元测试小计	**161 / 161**	31 个测试类

`tests/http_retrieval_acceptance.py`	**236 / 236 项**	39 个用例，覆盖本变更全部 Scenario 的正向与反向判据
`tests/test_acceptance.py`	**23 / 23 OK**	既有能力场景级回归
`tests/http_acceptance.py`	**58 / 58 项**	既有能力真实 HTTP 回归
`scripts/validate_spec.py`（本变更 + 覆盖检查）	**0 error / 0 warning**	规约结构与任务覆盖校验
`scripts/check_compose.py`	**14 / 14 项**	Docker 交付物静态校验

本轮复跑时修掉的两个脚手架缺陷

这两处都**不是功能缺陷**，而是 Windows 控制台编码造成的「看起来像缺陷」：

- `tests/http_retrieval_acceptance.py` 调用 `backfill_index.py` 时，子进程按本机区域编码（GBK）写 stdout，而父进程固定按 UTF-8 解析，抛出 `UnicodeDecodeError`；读线程崩溃使返回值为空，`backfill-idempotent` 的 **5 项判据被判失败**——库里其实一切正常。**修法**：为子进程注入 `PYTHONIOENCODING=utf-8`，并把解码改为 `errors="replace"`。修复后该项恢复通过。
- `scripts/check_compose.py` 输出 `✓ / ✗` 时在 GBK 控制台抛 `UnicodeEncodeError`，脚本直接崩掉，表现为「校验崩溃」而非「交付物不合格」。**修法**：脚本启动时把 stdout 重配置为 UTF-8。

两条都归到同一类问题：**脚本不该让输出编码决定自己的成败。**现已一同加固。

6.3 结论边界

以下是必须讲清楚的边界，避免把「等价验证」误读成「真实环境已验证」。

项目	状态	说明
真实 Qdrant 集群	待复核	本机未部署 Qdrant。验收使用的是行为等价的 REST 子集替身（真算余弦、支持 pre-filter 与阈值），**真实 Qdrant 上的点写入、索引与检索仍需在目标环境复核一次。**
课程网关真实凭证	待复核	嵌入与对话两侧使用确定性伪嵌入替身与对话替身验证流程，真实网关的返回格式与维度需以 `EMBEDDING_DIM` 实测确认。
Docker Compose 端到端	待复核	本机无 Docker，交付物以静态校验通过；容器内 `down/up` 后的数据持久化与索引重建需在具备 Docker 的环境补做。
HTTPS 下的 Cookie 加固	待加固	见 §3.9 第 10、11 项。
其余全部结论	已验证	单元、真实 HTTP 验收、既有能力回归、规约校验均在本机实际执行。

替身验证的可信度来自哪里

替身不是「随便返回点东西」。伪嵌入是**确定性**的（同一文本恒得同一向量），向量库替身**真实计算余弦相似度**并真实执行 pre-filter 与阈值判定。因此被验证的是**本项目的逻辑**——班级前置过滤是否生效、阈值边界是否判定正确、RRF 名次是否融合正确——这些结论不会因为换成真实 Qdrant 而改变。真正需要复核的是**集成细节**：真实的返回结构与字段命名。

7 附录

7.1 环境变量

变量	必要性	说明
`SECRET_KEY`	**必填**	会话签名密钥；缺失或短于 16 字符则拒绝启动
`DATABASE_PATH`	可选	SQLite 文件路径
`UPLOAD_DIR`	可选	上传目录
`MAX_UPLOAD_BYTES`	可选	上传大小上限，默认 5 MiB
`EMBEDDING_BASE_URL`	可选	嵌入接口基址；缺失则向量模式降级
`EMBEDDING_API_KEY`	可选	嵌入密钥，仅服务端
`EMBEDDING_MODEL`	可选	嵌入模型名
`EMBEDDING_DIM`	可选	向量维度，须与网关返回一致
`CHAT_BASE_URL` / `CHAT_API_KEY` / `CHAT_MODEL`	可选	对话接口；留空则复用嵌入侧的基址与密钥
`QDRANT_URL`	可选	向量库地址
`QDRANT_COLLECTION`	可选	集合名，默认 `campusclaw_chunks`
`QDRANT_API_KEY`	可选	向量库密钥；自建未开鉴权时可留空

7.2 数据表结构

表	班级私有	用途
`classes`	—	班级
`users`	是（`class_id`）	用户；口令仅存 bcrypt 哈希
`materials`	是	材料元信息；含 `index_status` / `chunk_strategy` / `source_sha256`
`knowledge_entries`	是	知识条目；与材料同事务写入
`knowledge_chunks`	是	**检索核心表**：切片正文、偏移、行号、哈希、索引状态
`handouts` / `assignments` / `assistants` / `skills`	是	讲义、作业、助手、技能四类结构

索引：`users(class_id)`、`materials(class_id, created_at DESC)`、`knowledge_entries(class_id, material_id)`、`knowledge_chunks(class_id, material_id, chunk_index)`、`knowledge_chunks(material_id)`。班级条件出现在每个索引的前导列上，与「班级优先」的查询模式一致。

7.3 固化常量

常量	值	位置
`SESSION_KEYS`	5 个字段	`app/auth.py`
`MIN_SECRET_KEY_LEN`	16	`app/__init__.py`
`RRF_K`	60	`app/retrieval.py`
`VECTOR_MIN_SCORE`	0.35	`app/retrieval.py`
`DEFAULT_LIMIT` / `MAX_LIMIT` / `MAX_OFFSET`	10 / 20 / 1000	`app/retrieval.py`
`MAX_QUERY_CHARS`	300	`app/retrieval.py`
`VECTOR_CANDIDATES`	50	`app/retrieval.py`
`TOP_K_CONTEXT`	4	`app/qa.py`
`MAX_HISTORY_TURNS`	20	`app/qa.py`
`BATCH_SIZE`（嵌入）	64	`app/gateway.py`
`DEFAULT_MAX_CHARS` / `DEFAULT_OVERLAP_PERCENT`	800 / 10	`app/chunking.py`
`MIN_MERGE_CHARS`	32	`app/chunking.py`
`DEFAULT_COLLECTION`	`campusclaw_chunks`	`app/vector_store.py`

本文档由仓库中的实际代码与规约整理而成，未引入仓库之外的设计。所有常量、字段名与行为描述均可按 §7.3 给出的文件位置逐条核对。